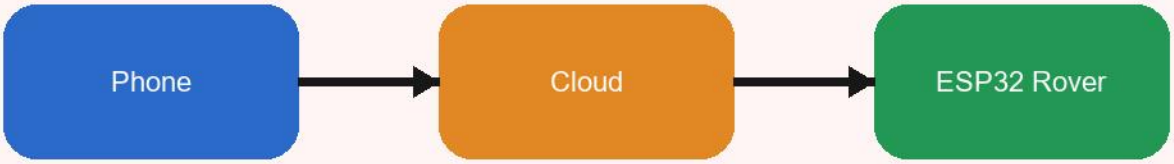
ESP32-S3 4WD Rover Manual

From first power-on to remote control from another internet connection

What this guide teaches

This PDF explains the full workflow from zero: how to use the rover through the ESP hotspot, how to connect it to your home router, how to publish the GitHub Pages phone website, how to create an MQTT cloud broker, how the current firmware pin map is wired, how power should flow through the full rover, and how to drive the rover while your phone and rover are on different internet connections.



The Three Real Ways You Will Use It

1. Phone connected directly to the ESP32 hotspot. Use this for first setup and recovery. The address is always http://192.168.4.1/
2. Phone and rover connected to the same home Wi-Fi router. Use the router IP shown by the rover page, for example http://192.168.1.23/
3. Phone far away from home while the rover stays at home on Wi-Fi. Use the GitHub Pages remote app plus the MQTT cloud broker.

Important URLs

These are the exact addresses you will use most often

ESP hotspot control page
http://192.168.4.1/

ESP hotspot status page
http://192.168.4.1/status

GitHub repository
https://github.com/Yanzoo-da/ESP3-4wd

GitHub Pages settings
https://github.com/Yanzoo-da/ESP3-4wd/settings/pages

GitHub Pages docs home
https://yanzoo-da.github.io/ESP3-4wd/

Remote phone control page
https://yanzoo-da.github.io/ESP3-4wd/remote-control/

HiveMQ console
https://console.hivemq.cloud

How To Think About These URLs

- Use 192.168.4.1 only when your phone is connected to the ESP32 hotspot.
- Use the router IP only when your phone and rover are on the same home Wi-Fi.
- Use the GitHub Pages remote app when your phone is away from home and on a different internet connection.
- The remote app does not connect to the rover directly. It talks to the MQTT cloud broker, and the rover talks to that same broker.
- The router IP is not fixed unless you reserve it in your router.

Current Project Values

The exact GitHub and HiveMQ values currently used by this rover project

GitHub Values

Repository
https://github.com/Yanzoo-da/ESP3-4wd

Pages settings
https://github.com/Yanzoo-da/ESP3-4wd/settings/pages

Pages home
https://yanzoo-da.github.io/ESP3-4wd/

Remote control page
https://yanzoo-da.github.io/ESP3-4wd/remote-control/

HiveMQ And Broker Values

HiveMQ console
https://console.hivemq.cloud

Broker host
8038be31051d4e368ce62a5753aaf95d.s1.eu.hivemq.cloud

ESP TLS port
8883

Browser WebSocket port
8884

WebSocket path
/mqtt

MQTT username
Yanzoo4wd

Topic base
rover/yanzoo-car-1

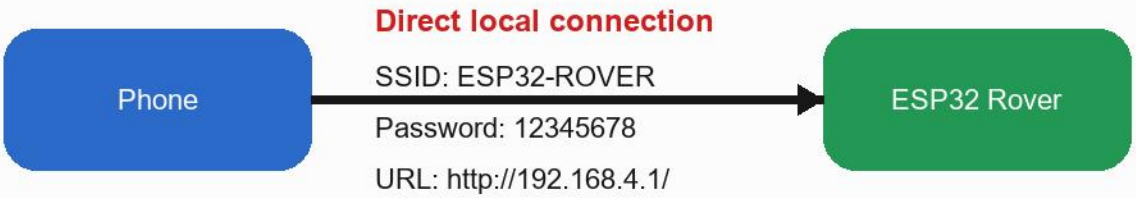
Broker password is intentionally not written into this PDF.

Enter it manually on the ESP local page and on the GitHub Pages remote page.

Step 1: Use The ESP32 Hotspot First

This is the safest and simplest first test

What The Connection Looks Like



Exact Steps

1. Power on the rover and wait about 20 to 30 seconds.
2. Open Wi-Fi settings on your phone and connect to ESP32-ROVER.
3. Enter the password 12345678.
4. Open your phone browser and go to http://192.168.4.1/
5. If you want raw status information, open http://192.168.4.1/status and look at mode, stationIp, apIp, wifiStatus, and message.
6. Use Buttons or Joystick to test local movement with the wheels raised at first.
7. If this page works, your ESP web server is working correctly.

Step 2: Connect The Rover To Your Home Wi-Fi

Do this from the local rover page while the phone is on the ESP hotspot

Exact Steps

1. While still connected to the ESP hotspot, open http://192.168.4.1/
2. Scroll to the section named Wi-Fi And Hotspot.
3. Type your home router Wi-Fi name in the SSID field exactly as it appears on the router.
4. Type your home router password in the password field exactly.
5. Press Save And Connect.
6. Wait up to about 30 seconds while the ESP tries to join the router.
7. If it works, the status area should show something like Network: STA+AP ready and Router YourWiFiName @ 192.168.1.23
8. Write down that router IP because that is the address you will use when your phone is also on the same router.
9. After the first successful save, the local ESP page can auto-fill the stored Wi-Fi SSID and password on later visits because they are saved in the board preferences storage.
10. If it fails, the page usually stays in AP fallback mode and the router IP remains 0.0.0.0.
11. If wifiStatus shows WL_CONNECT_FAILED, the password is usually wrong. If it shows WL_NO_SSID_AVAIL, the SSID is wrong, hidden, or the 2.4 GHz router band is not available.

Example same-router URL after success

http://192.168.1.23/

Step 3: Use The Rover On The Same Home Wi-Fi

This is normal local home use after the rover joins your router

What The Connection Looks Like

Exact Steps

1. Disconnect your phone from ESP32-ROVER.
2. Connect your phone to your normal home Wi-Fi.
3. Open the router IP that the rover showed earlier, for example http://192.168.1.23/
4. If the page opens, you now control the rover through your home router instead of the hotspot.
5. If it does not open, reconnect to ESP32-ROVER and check whether the router IP changed or whether the rover fell back to hotspot mode.
6. This same-router mode is the best place to test driving, speed control, RGB effects, and Auto Avoid before internet control.

Step 4: Publish The Phone Website With GitHub Pages

This makes the remote browser app available on the internet

Exact GitHub Pages Setup

1. Open your repository: https://github.com/Yanzoo-da/ESP3-4wd
2. Log in to GitHub.
3. Open the Pages settings directly: https://github.com/Yanzoo-da/ESP3-4wd/settings/pages
4. Under Build and deployment, find Source.
5. Choose Deploy from a branch.
6. For the branch, choose main.
7. For the folder, choose /docs.
8. Click Save.
9. Wait a few minutes. GitHub says publication can take up to about 10 minutes.
10. Then open https://yanzoo-da.github.io/ESP3-4wd/
11. Your remote control page should be https://yanzoo-da.github.io/ESP3-4wd/remote-control/
12. If you see a 404 page, wait a little longer, then verify that the Pages source is main and /docs.

Remote app URL after Pages is live

https://yanzoo-da.github.io/ESP3-4wd/remote-control/

Step 5: Create The MQTT Cloud Broker

Use HiveMQ Cloud so the rover and your phone can meet over the internet

Exact HiveMQ Setup

1. Open https://console.hivemq.cloud
2. Create an account using GitHub, Google, LinkedIn, or email.
3. After sign-in, choose Start in HiveMQ Cloud.
4. Click Create Cloud Cluster.
5. Choose HiveMQ Cloud Serverless.
6. Create the serverless cluster and wait until it becomes ready.
7. Open Manage Cluster.
8. On the cluster overview page, write down the cluster URL, MQTT TLS port, WebSocket port, and the secure WebSocket URL.
9. For this project, the ESP usually uses the TLS MQTT port, often 8883, and the phone web app usually uses the secure WebSocket port, often 8884.
10. If your cluster shows different values, use the values from your own cluster overview instead of assuming 8883 and 8884.

What You Must Write Down

Broker host | TLS port | WebSocket port | WebSocket path

Step 6: Create Broker Credentials

The rover and your phone remote app must both use the same broker login

Exact Credential Steps

1. Inside the HiveMQ cluster, open Access Management.
2. Open the credentials area and choose Add Credentials.
3. Create a username, for example roveruser.
4. Create a strong password.
5. Give the credential permission to publish and subscribe.
6. Save the credential.

Pick One Topic Base And Keep It Identical

Now choose one topic base for your rover. Example: rover/yanzoo-car-1. The rover and the remote app must use the exact same topic base. If one character differs, the remote app may connect to the broker but the rover will not react.

Example topic base

rover/yanzoo-car-1

Example WebSocket path

/mqtt

Step 7: Save MQTT Settings Into The Rover

Do this on the rover's local web page before trying remote control

Fields To Fill On The Rover Page

1. Open the rover local page, either http://192.168.4.1/ or the rover's router IP.
2. Open Cloud MQTT Remote Control.
3. Broker host: paste the HiveMQ cluster URL.
4. ESP TLS port: use the MQTT TLS port from HiveMQ, usually 8883.
5. WebSocket port: use the secure WebSocket port from HiveMQ, usually 8884.
6. WebSocket path: use /mqtt unless your cluster documentation says otherwise.
7. MQTT username: enter the broker username you created.
8. MQTT password: enter the broker password you created.
9. Topic base: use the project topic base rover/yanzoo-car-1 unless you intentionally want a different one.
10. Press Save Cloud MQTT and wait a few seconds.
11. If it works, the rover page should show Cloud: connected.
12. The GitHub Pages remote app now uses the same project topic base by default, so the hotspot page, router page, and cloud page can stay aligned.

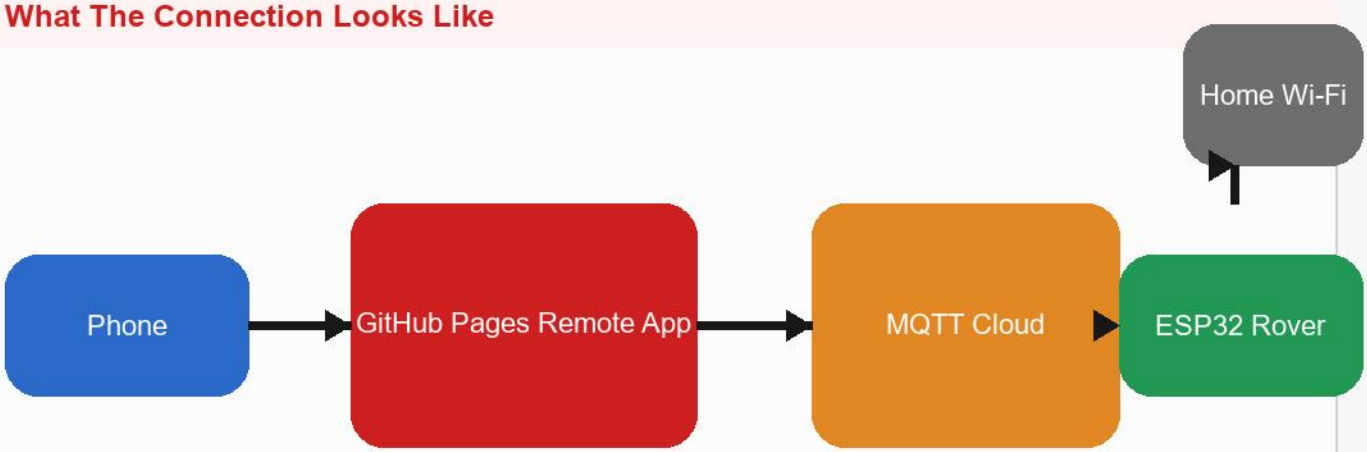
If Cloud stays disconnected

Check host, ports, username, password, and topic base.

Step 8: Control The Rover From Another Internet

This is the away-from-home workflow

What The Connection Looks Like



Exact Remote Steps

1. Leave the rover at home powered on.
2. Make sure the rover is connected to your home Wi-Fi.
3. Make sure the rover local page already shows Cloud: connected.
4. Leave home or switch your phone to mobile data or another Wi-Fi.
5. Open the remote app: https://yanzoo-da.github.io/ESP3-4wd/remote-control/
6. Enter the same broker host, WebSocket port, WebSocket path, username, password, and topic base that you used on the rover.
7. Press Connect Broker.
8. If the app shows Broker: connected, press Refresh Rover Status.
9. If rover telemetry appears, remote control is ready. Test Stop first, then very slow movement.

How Remote Control Works

Why it still works even when phone and rover are on different internet connections

Important Rules

- When you are away from home, do not try to use the rover's private router IP like 192.168.1.23. That IP only works inside your home network.
- When you are away, use https://yanzoo-da.github.io/ESP3-4wd/remote-control/
- The remote app does not directly reach your home router. It reaches the MQTT broker on the internet.
- The rover also reaches that same MQTT broker on the internet.
- Because both sides open outgoing connections to the broker, they can communicate even when they are on different networks.
- If the remote app connects but the rover does not react, the most common cause is that the topic base on the phone and rover do not match exactly.
- If the rover loses home Wi-Fi and falls back to hotspot mode, remote internet control stops until the rover gets internet again.
- If the office Wi-Fi blocks secure WebSockets, the remote app may fail there but still work over phone mobile data.
- Without a camera feed, remote driving is risky. Start with low speed and safe test conditions.

Parts Used By Firmware vs Build Only

What from your shopping list is actually represented in code

Used In Code

- ESP32-S3-N16R8 development board
- Three HC-SR04 ultrasonic sensors
- Two L298 motor driver modules
- Four DC geared motors
- 4WD rover platform logic
- RGB WS2812 user LED

Used In Wiring / Build

- 3 x 18650 cells and battery holder
- 3S BMS, fuse, switch, DC connectors
- XL4015 buck converters and AMS1117 module
- Capacitors, resistors, jumper wires, breadboard
- Solder, heatsink, mechanical chassis parts

Tools / Not Firmware Parts

- Multimeter, screwdriver, HDMI cable, and spirit level are useful tools or accessories, but they are not firmware-controlled parts.
- The code is ready for the main controllable electronics, but the power and assembly parts belong to the wiring plan rather than to src/main.cpp.

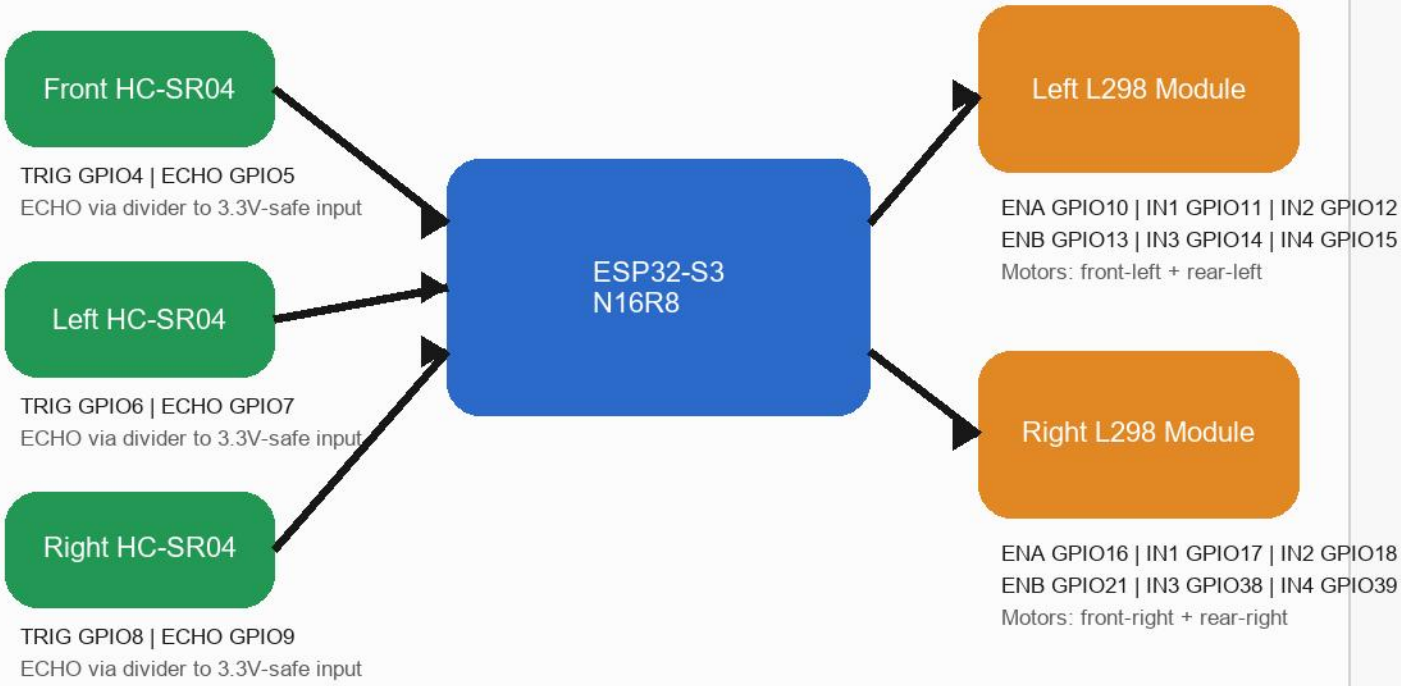
Important Note

Even for the parts already used in code, the final hardware still depends on your real wiring matching the GPIO pin assignments in src/main.cpp.

Project Pinout Diagram

Current firmware GPIO map for the ESP32-S3 N16R8 rover reference build

Main Pinout Diagram



Onboard RGB LED

WS2812 / NeoPixel on GPIO48

Reserved ESP32-S3 pins

Do not use GPIO35, GPIO36, or GPIO37 on this N16R8 board.

Pinout Notes

- The motor GPIO map above matches the current src/main.cpp firmware and the project PDF should be kept in sync with that file.
- Each HC-SR04 ECHO signal must be reduced to a 3.3V-safe level before it reaches the ESP32-S3 input pin.
- The ESP32 onboard WS2812 LED is already handled by firmware and does not need separate control wiring.

Exact Wiring Reference

What connects to what in the current rover firmware layout

Left Side Drive

- L298 module A is the left-side driver reference.
- Front-left motor connects to output channel A on the left L298.
- Rear-left motor connects to output channel B on the left L298.
- Left L298 ENA -> GPIO10
- Left L298 IN1 -> GPIO11
- Left L298 IN2 -> GPIO12
- Left L298 ENB -> GPIO13
- Left L298 IN3 -> GPIO14
- Left L298 IN4 -> GPIO15

Right Side Drive

- L298 module B is the right-side driver reference.
- Front-right motor connects to output channel A on the right L298.
- Rear-right motor connects to output channel B on the right L298.
- Right L298 ENA -> GPIO16
- Right L298 IN1 -> GPIO17
- Right L298 IN2 -> GPIO18
- Right L298 ENB -> GPIO21
- Right L298 IN3 -> GPIO38
- Right L298 IN4 -> GPIO39

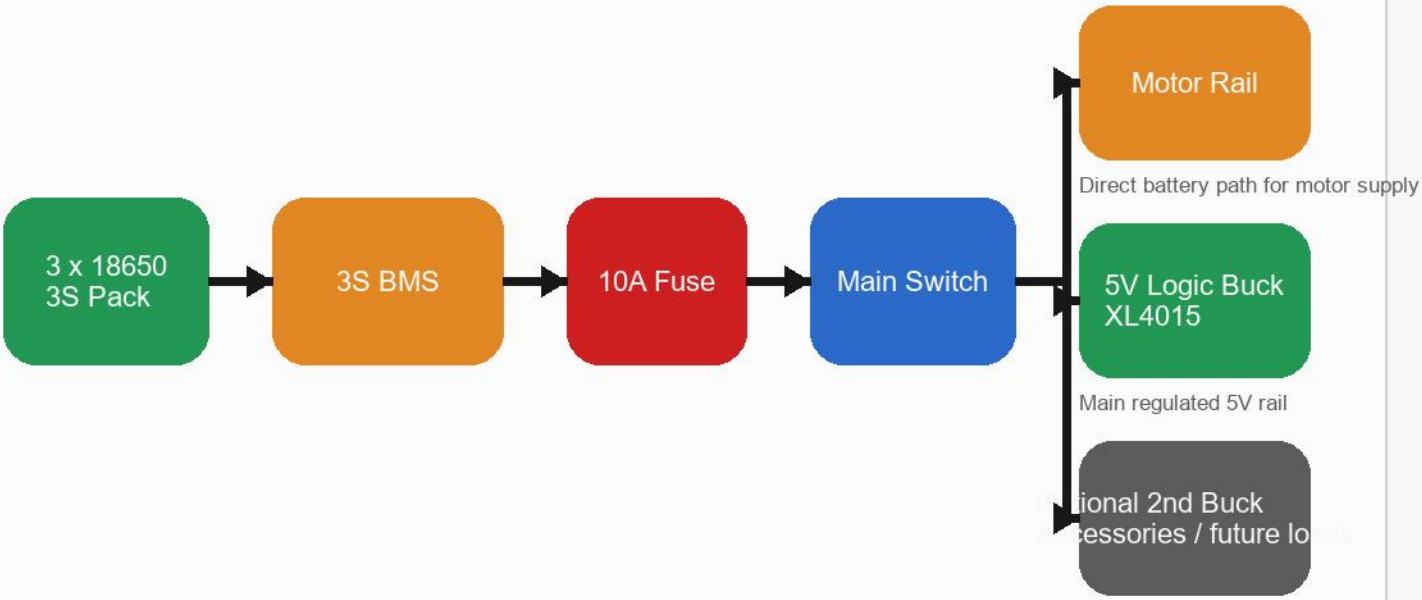
Sensors And Support Wiring

- Front HC-SR04: TRIG -> GPIO4, ECHO -> GPIO5 through a voltage divider, VCC -> 5V rail, GND -> common ground.
- Left HC-SR04: TRIG -> GPIO6, ECHO -> GPIO7 through a voltage divider, VCC -> 5V rail, GND -> common ground.
- Right HC-SR04: TRIG -> GPIO8, ECHO -> GPIO9 through a voltage divider, VCC -> 5V rail, GND -> common ground.
- Recommended ECHO divider examples: 10k from ECHO to GPIO and two 10k resistors in series from GPIO to GND, or any equivalent 5V-to-3.3V divider.
- Place a 1000uF capacitor near the motor supply rail and 100nF decoupling close to logic rails where practical.
- All grounds must join: battery, BMS output, buck converters, ESP32, both L298 modules, and all sensors.

Full Project Power Flow Diagram

Reference power architecture using the parts already purchased

Recommended Power Flow



What Connects To The Motor Rail

- Motor supply input of the left L298 module
- Motor supply input of the right L298 module
- Large bulk capacitor near the driver supply rail
- This path carries the noisiest current and should use short, heavier wiring

What Connects To The 5V Logic Rail

- ESP32-S3 board 5V / VBUS input or short USB power lead
- All three HC-SR04 VCC pins
- Optional external logic 5V input for L298 modules if your board and jumpers support it
- Future light accessories or low-current logic loads

Important: do not use AMS1117 directly from the 3S battery as the main ESP supply. Use a buck converter for the 5V rail.

Controls And Troubleshooting

What each control means and what to check when something fails

Main Controls

- Buttons: hold a direction button to keep moving.
- Joystick: drag to steer and release to stop.
- Speed slider: sets manual speed.
- Start Auto Avoid: enables autonomous obstacle avoidance.
- Manual Mode: returns control to the phone.
- Stop Auto: exits auto and stops the rover.
- RGB panel: choose color or police effect.
- Wi-Fi section: save or clear router credentials.
- Cloud MQTT: save or clear remote broker settings.

Quick Fixes

- Hotspot not visible: wait 30 seconds and scan again.
- Hotspot page not opening: type http://192.168.4.1/ exactly.
- Router page not opening: the rover may not be on Wi-Fi anymore.
- Pages URL gives 404: Pages may not be enabled yet or still publishing.
- Remote app connects but rover does nothing: check topic base, WebSocket port, and broker login on both sides.
- Wheels do not move: confirm L298 wiring and GPIO mapping.
- Sensors read badly: recheck ultrasonic wiring and ECHO voltage compatibility.
- When confused, go back to the simplest path: hotspot mode, local page, manual stop, low speed.